

İçindekiler

Çalışma Kılavuzu

BÖLÜM 1 — Terimler

Doküman ve girdi terimleri

Teknik terimler

Süreç terimleri

BÖLÜM 2 — Yeni projeye başlamadan önce

2.1 Hazırlık

2.2 Cevaplarını önceden düşün

BÖLÜM 3 — Kurulum: /yeni-proje

BÖLÜM 4 — Kurulumdan sonra: bir oturum nasıl geçer

Bir oturumun ritmi

★ Yorumlar neden bu kadar ayrıntılı — ikinci bir sebebi var

Neden her adımda /clear

BÖLÜM 5 — Hangi dosya ne işe yarıyor

Kök dizin

★ "Kendi değerlerini yazar" ne demek — aynı ayarın ÜÇ farklı değeri olur

Bir projede hangi ayarlar çıkar — gerçek liste

★ Bir değişken eklendiğinde ne olması gerekiyor — dört yer

docs/standards/ — her projede aynı

docs/project/ — her projede var, içeriği projeye özel

BÖLÜM 5B — Canlıya çıkış: üç yol ve neyle yapıldıkları

Yol A — Yönetilen servisler (kendi projelerinde varsayılan)

Yol B — Kendi sunucun (alan adı + AWS/Hetzner + Docker Engine)

Yol C — Kurumun kendi sunucusu (işyeri projelerinde en olası)

🚫 DEVOPS SINIRI — işe başlamadan sorulacak ÜÇ SORU

Hangisini ne zaman

BÖLÜM 6 — Hangi komut ne zaman

BÖLÜM 7 — Takıldığında

BÖLÜM 8 — Öğrendiklerim defteri

Kim yazar

Ne nereye gider

Ne işe yarıyor

★ Ajan seviyeni takip eder — dört seviyeli defter

BÖLÜM 9 — Bu kılavuz nasıl kısaltılır

Çalışma Kılavuzu

Bu dosya kullanıcı içindir. Ajanın uyacağı kurallar `CLAUDE.md` ve `docs/standards/` içinde; burada projenin nasıl yürütüleceği anlatılıyor.

★ **Bölüm 1 (terimler) zamanla silinebilir.** Terimler oturduğunda o bölümü kaldır, kılavuz kısalsın. Diğer bölümler kalıcıdır.

BÖLÜM 1 — Terimler

Akış boyunca geçen kelimeler. Bilinmeyen bir terim, verilen cevabı da geçersiz kılar.

Doküman ve girdi terimleri

Terim	Ne demek	Somut örnek
Analiz dokümanı	Projenin ne yapması istendiğini anlatan, sana verilen belge	<code>analiz.docx</code> , şartname, ihale dosyası, talep formu
PRD (Product Requirements Document)	"Sistem tam olarak ne yapacak" sorusunun yazılı ve eksiksiz hâli. Analiz dokümanından üretilir	Roller, iş kuralları, kapsam dışı, hata durumları
API dokümanı	Var olan bir servisin nasıl çağrılacağını anlatan belge	Aşağıda açıldı
Veritabanı şeması	Var olan bir veritabanında hangi tablolar ve ilişkiler olduğunu gösteren belge	Aşağıda açıldı
ADR (Architecture Decision Record)	Önemli bir teknik kararın neden alındığını yazan kısa belge	"Repository Pattern kullanmadık, çünkü..."
Roadmap	Yapılacak adımların sırayla listesi, kutucuklu	"Adım 4 — kimlik doğrulama 

⚠ **Analiz dokümanı ile PRD karıştırılmaz.** Analiz **girdidir**, sana verilir ve eksiktir. PRD **çıktıdır**, sorular sorularak üretilir ve eksikliği kalmaz.

Bu üç belge fiilen neye benziyor

Analiz dokümanı genelde Word veya PDF. İçinde: projenin amacı, kimlerin kullanacağı, hangi ekranların olacağı, hangi kuralların işleyeceği.

⚠ **Her zaman eksiktir.** "*İş emri kapatılabilmelidir*" yazar ama "*kim kapatabilir, hangi durumdayken, açıklama zorunlu mu*" yazmaz. PRD görüşmesi tam da bu boşlukları kapatmak içindir.

API dokümanı, var olan bir servisi kullanacaksan gerekir. Üç şeyi söyler: hangi adrese istek atılacak, ne gönderilecek, ne dönecek.

```
GET /personel/{id}
```

```
Dönen: { "ad": "...", "soyad": "...", "birim": "...", "durum": "aktif" }
```

Genelde Swagger/OpenAPI sayfası, Postman koleksiyonu veya bir Word dosyası olarak gelir. **Yoksa** kurumdan istenir; onsuz o servise bağlanılamaz.

Veritabanı şeması, var olan bir veritabanına bağlanacaksan gerekir. Hangi tablolar var, içlerinde hangi kolonlar, tablolar birbirine nasıl bağlı.

Tablo: personel

id (sayı, birincil anahtar)

ad (metin)

birim_id (sayı) → birim tablosuna bağlı

Kutucuk-ok çizimi (ERD), tablo listesi veya **CREATE TABLE** komutları hâlinde gelebilir.

⚠ **Üçü de yoksa sorun değil** — yeni bir sistem kuruyorsan zaten olmaz. Kit soru sorarak eksiği tamamlar.

Teknik terimler

Terim	Ne demek
İstemci / tüketici	API'den veri çeken program: web arayüzü, mobil uygulama, başka kurumun sistemi
Uç nokta (endpoint)	Uygulamanın dışarıya açtığı bir adres: " <i>buraya şunu gönderirsen şunu yaparım</i> "
Migration	Veritabanının yapısını değiştiren komut dosyaları — aşağıda açıldı
Seed	Geliştirme için üretilen sahte örnek veri
Konteyner	Uygulamayı ihtiyaçlarıyla birlikte paketleyip çalıştıran kutu (Docker)
Ortam değişkeni	Koda yazılmayan, dışarıdan verilen ayar — aşağıda açıldı
CI	Kod gönderildiğinde otomatik çalışan test ve derleme zinciri
Deploy	Kodun, kullanıcıların erişebildiği bir yerde çalışır hâle getirilmesi
İzleme (monitoring)	Sistem canlıdayken neyin yavaşladığını, neyin hata verdiğini gösteren araçlar

Migration nedir — açık hâli

Veritabanının **yapısı** değiştiğinde (yeni tablo, yeni kolon, silinen kolon) bu değişiklik elle yapılmaz. Bunun yerine bir **dosya** üretilir.

Dosya nerede durur: `prisma/migrations/` klasöründe, tarih damgalı:

```
prisma/migrations/  
├─ 20260101120000_ilk_tablolar/migration.sql  
├─ 20260115093000_is_emrine_foto_alani_ekle/migration.sql  
└─ 20260203141500_lokasyona_index_ekle/migration.sql
```

"Sırayla çalışır" ne demek: Üçüncü dosya, ilk ikisinin çalıştığını varsayar. "Fotoğraf alanı ekle" komutu ancak tablo daha önce oluşturulmuşsa anlamlıdır. Bu yüzden dosya adları tarih damgalı — sıra kaybolmasın diye.

"Kayıt altına alınır" ne demek: İki şey birden:

1. Dosyalar **git'e girer**, yani sen, DevOps ve otomatik testler **aynı** komutları çalıştırır. "*Bende tablo var sende yok*" durumu oluşmaz
2. Veritabanı kendi içinde **hangilerinin çalıştığını** bir tabloda tutar. Komut ikinci kez çalıştırılmaz

Sen ne yapıyorsun: Şemayı değiştirip komutu veriyorsun; dosya kendiliğinden üretiliyor. SQL yazmıyorsun.

⚠ Canlı ortamda yalnızca "*uygula*" komutu çalışır, "*üret*" komutu asla — ikincisi veri kaybettirebilir.

Ortam değişkeni nedir — açık hâli

Aynı kod farklı yerlerde farklı ayarlarla çalışır: senin bilgisayarında başka veritabanı, canlıda başka. Bu ayarlar **koda yazılmaz**, dışarıdan verilir.

Nereye yazılır: Proje kökündeki `.env` dosyasına, `AD=değer` biçiminde:

```
DATABASE_URL=postgres://kullanici:sifre@localhost:55432/bakim  
JWT_SECRET=uzun-rastgele-bir-metin  
WEB_PORT=3100
```

Kim okur: Uygulama açılırken bu dosyayı okur ve değerleri alır.

Neden koda yazılmıyor — iki sebep:

1. **Değer ortama göre değişir.** Senin makinende `localhost`, canlıda kurumun sunucusu. Kod aynı kalmalı, yalnızca değer değişmeli
2. 🚫 **İçinde şifre var.** Koda yazılırsa git'e girer; git geçmişisi silinmez, yani bir kez girdiyse **sonsuz kadar orada kalır**

Süreç terimleri

Terim	Ne demek
Dal (branch)	Ana koda dokunmadan çalışılan kopya
Birleştirme isteği (PR / MR)	"Bu dalı ana koda alalım mı" önerisi; inceleme burada yapılır
Oturum (session)	Ajanla yapılan bir çalışma turu. Bir roadmap adımı = bir oturum
Devir notu	Oturum kapanırken yazılan "sırada ne var" notu

BÖLÜM 2 — Yeni projeye başlamadan önce

2.1 Hazırlık

1. **Boş bir klasör** aç ve VS Code'da aç
2. **Kiti güncelle** — her yeni projede, atlanmaz. Sohbeta yapıştır:

```
claude plugin update proje-kiti@bariskose-skills
```

Eklentinin tam adı **proje-kiti@bariskose-skills** — **proje-kiti** eklenti adı, **bariskose-skills** ise geldiği kaynak.

Sonra **pencereyi yenile** — yoksa eski sürüm çalışmaya devam eder: üst menüde **View** → **Command Palette...** → kutuya **Reload Window** yaz → çıkan **Developer: Reload Window** satırına tıkla. (Klavyeyle: **Cmd+Shift+P** / **Ctrl+Shift+P**)

⚠ Kite sürekli yeni kural ekleniyor. Eski sürümle başlarsan o kurallar projeye hiç gelmez.

3. **Elindeki belgeleri klasöre koy** — varsa:

Belge	Ne zaman gerekir	Yoksa
Analiz dokümanı	Her zaman — projenin ne yapacağı burada	Sözlü anlatırsın, kit yazıya döker
API dokümanı	Var olan bir servise bağlanacaksan	Yeni sistemde gerekmez
Veritabanı şeması	Var olan bir veritabanına bağlanacaksan	Yeni sistemde gerekmez

⚠ Bu belgeler **girdidir**; sen yazmazsın, sana verilir. İçeriklerinin neye benzediği Bölüm 1'de açıklandı.

2.2 Cevaplarını önceden düşün

Kurulumda şunlar sorulacak. Şimdiden düşünmek görüşmeyi hızlandırır:

Soru	Ne cevaplayacaksın
Bu proje kimin için?	İşyeri projesi mi, kendi projen mi
Proje tipi?	Web · mobil · ikisi
Yazdığın API'yi, projenin dışındaki bir sistem tüketecek mi?	Başka kurum, yüklenici, merkezî sistem. 🚫 Kendi web arayüzün ve kendi mobil uygulaman bu soruya ""hayır"" dır — ikisini de sen yazıyorsun, ne isteyeceklerini biliyorsun
Kendiliğinden çalışması gereken iş var mı?	Zamanlanmış görev, gece raporu
Kod nerede duracak, deploy'u kim yapacak?	GitHub/GitLab · sen/DevOps
Canlıya nasıl çıkacak?	Üç yol var — BÖLÜM 5B'de açıldı. İşyeri projesinde bu soru sana sorulmaz , kurumun DevOps ekibi karar verir

BÖLÜM 3 — Kurulum: **/yeni-proje**

Tek komut:

```
/yeni-proje
```

Gerisi sohbet. Sırayla şunlar olur:

Adım	Ne oluyor	Senden ne isteniyor
0	Eklentiler kontrol edilir, platform tespit edilir, ses bildirimi kurulur	İzin
1	Kim için · proje tipi · backend kurgusu (4 soru) · API biçimi (4 soru) · proje adı	Cevaplar
2	Kit dosyaları projeye kopyalanır	—
3	★ PRD görüşmesi — en uzun adım	Analiz dokümanını verirsın, tek tek soru cevaplırsın
4	Yol haritası, ilk kararlar, teknoloji-ve-plan iskeleti	Onay
5	İskelet kurulur — ilk kod	Onay
6	Yayın veya teslim paketi (1. adımdaki cevaba göre)	Duruma göre
7	Son kontrol	—

⚠ **Bu tek komutluk bir işlem değil.** `/yeni-proje` kurulumu başlatır; Adım 3 uzun bir görüşmedir. Vaat "tek promptla uygulama" değil, "doğru kurulmuş proje ve net yol haritası".

⊖ **Adım 3'te acele etme.** Analiz dokümanında yazmayan onlarca karar orada netleşir. Cevabı bilinmeyen bir kural kodlanırsa yanlış varsayım tüm katmanlara yayılır.

BÖLÜM 4 — Kurulumdan sonra: bir oturum nasıl geçer

Kurulum bitince artık kit değil, projedeki `CLAUDE.md` ve `docs/standards/` geçerlidir. İlerleme **roadmap adımlarıyla** olur.

Bir oturumun ritmi

1. Aç ve devral → "docs/project/sonraki-adim-prompt.md oku, devam edelim"
2. Plan sun → ajan ne yapacağını anlatır
3. Onayla → gerekiyorsa düzelt
4. Kod + test → ajan yazar, testler yeşil olur
5. Gözle doğrula → ekran varsa tarayıcıda görülür
6. Ajan denetimi → ★ ajan kendi yazdığını inceler, bulguları düzeltir
7. Commit + öneri → değişiklik önerisi açılır
8. Kutucuk ✅ → roadmap'te o adım işaretlenir
9. Kararları yaz → teknoloji-ve-plan.md güncellenir
10. Devir notu → sırada ne var yazılır
11. /clear → yeni oturuma temiz başla

🚫 **8 ve 9 atlanmaz.** Kutucuk *nerede kalındığını*, teknoloji belgesi *neden öyle yapıldığını* söyler. İkisi de sonradan hatırlanmaz.

★ 6. adım — "testler yeşil" neden yetmiyor

Otomatik testler makinenin **ölçebildiğini** ölçer. Ölçemedikleri var ve gerçek hatalar çoğu zaman oralarda:

Test yakalar	🚫 Test yakalayamaz
Test kırmızı mı	Test yanlış şeyi doğruluyor mu
Tip hatası	Tip doğru ama iş kuralı yanlış
Katman ihlali	Katman temiz ama sorumluluk yanlış yerde
—	Yorumlar eksik veya yanlış
—	Yeni bir güvenlik açığı

Bu yüzden ajan, değişiklik önerisi açmadan **önce kendi yazdığını denetler**: kod incelemesi · güvenlik denetimi · test kalitesi · yorumların yeterliliği. Bulgu çıkarsa **düzeltilir**, sonra sana sunulur.

🚫 **Sen bir şey yapmıyorsun** — bu ajanın kendi kapısı. Ama bilmen gerekiyor: "*testler geçti*" dediğinde iş bitmiş değildir, **bir kapı daha var**.

★ Yorumlar neden bu kadar ayrıntılı — ikinci bir sebebi var

Yorumların ilk sebebi belli: **sen, denetçi ve devralan geliştirici** kodu okumadan anlayabilsin.

Ama ikinci bir sebebi daha var ve o **doğrudan senin cebine dokunuyor**.

Sorun: yorumlu dosya 4 kat büyük

Ölçüldü: aynı kod, yorumlu hâlde **4.1 kat** daha uzun. Ajan onu okurken 4 kat fazla **jeton (token)** harcar. Bir oturumda on dosya okunuyorsa bu gerçek bir maliyet.

⚠️ (Kullanıcının indirdiği dosyaya etkisi **sıfır** — derleyici yorumları siler. Maliyet yalnızca ajanın okumasında.)

Çözüm: yorumlar bir HARİTA oluşturuyor

Her dosyanın başındaki blok sabit biçimde yazılıyor:

```
/**
 * İŞ EMRİ OLUŞTURMA
 *
 * NEREDEN : apps/web/.../talep-formu.tsx → kullanıcının doldurduğu form
 * NE      : üç kural sırayla uygulanıyor
 * NEREYE  : prisma → PostgreSQL "WorkOrder" tablosu · BullMQ kuyruğu
 * SONUÇ   : IE-2026-000148 numaralı kayıt oluşur, ekranda listede belirir
 */
```

★ **NEREDEN** ve **NEREYE** satırlarında gerçek dosya adı geçiyor. Bu, tüm yorumları **aranabilir bir bağımlılık haritasına** çeviriyor.

Gerçek hayat benzetmesi: Depodaki her kolinin üstünde "A deposundan geldi → B mağazasına gidecek" etiketi var. Kolileri **açmadan**, yalnızca etiketleri okuyarak tüm dağıtım ağını çıkarabilirsiniz.

Aramada bulunan	Anlamı
NEREYE ... <dosya>	O dosya buraya gönderiyor
NEREDEN ... <dosya>	O dosya buradan alıyor

★ İki yönlü grafik — hiçbir dosyanın **gövdesini açmadan**.

★ SEN DE ARAYABİLİRSİN — VS Code'da nasıl

Bu harita ajana özel değil. **Sen de tek kısayolla aynı sonucu alırsın** — "şu dosyayı değiştirsem ne bozulur" sorusunu bana sormadan cevaplayabilirsin.

Kısayol	Ne yapar
Cmd+F (Mac) · Ctrl+F (Win)	🚫 Yalnızca açık dosyada arar — yetmez
★ Cmd+Shift+F · Ctrl+Shift+F	Tüm projede arar — aradığın bu

Fareyle: Sol kenar çubuğunda **büyüteç** ikonu (Search).

Adım adım — örnek

Diyelim `work-orders.service.ts` dosyasını değiştireceksin ve "bu neyi bozar" diye merak ediyorsun.

1) Kim buna veri gönderiyor / bundan veri alıyor:

`Cmd+Shift+F` → arama kutusuna yaz:

```
work-orders.service
```

Çıkan sonuçlarda **NEREDEN** veya **NEREYE** satırlarına bak:

Sonuçta gördüğün	Anlamı
NEREYE : ...work-orders.service...	O dosya buraya gönderiyor
NEREDEN : ...work-orders.service...	O dosya buradan alıyor

2) Bulduğun dosyaların yalnızca başlık bloğunu oku — dosyayı aç, en üstteki `/** ... */` bloğunu oku, gerisini geç. Genelde **SONUÇ** satırı sorunu cevaplar: "bu bozulursa ekranda ne olmaz."

★ **Aramayı daraltmak istersen** kutuya şunu yaz — yalnızca yorum satırları gelir:

```
NEREYE.*work-orders.service
```

⚠ Bunun çalışması için arama kutusundaki **.*** düğmesi (Use Regular Expression) açık olmalı — kutunun sağındaki üçüncü küçük ikon.

Ne zaman işine yarar

Durum	Ne aratırsın
"Bu dosyayı silsem ne olur"	Dosya adı — kimse NEREDEN / NEREYE demiyorsa kimse bağımlı değildir
"Bu tabloya kim yazıyor"	Tablo adı, örn. <code>WorkOrderHistory</code>
"Bu ekran verisini nereden alıyor"	Ekranın dosya adı → başlık bloğundaki NEREDEN
"Şu kural nerede uygulanıyor"	Kuralın Türkçe adı, örn. <code>pasif lokasyon</code>

Üç adımlı tarama — ajan böyle çalışıyor

#	Soru	Nasıl
1	Bu dosya neyi etkiliyor	Kendi başlık bloğundaki NEREYE satırı
2	Buna kim bağımlı	Tüm depoda NEREDEN / NEREYE araması
3	Bulunanlar ne yapıyor	🚫 Sadece başlık bloklarını oku, gövdeyi değil

★ **Kazanç:** 200 satırlık bir dosyanın başlık bloğu ~15 satır. On dosyanın etkisini görmek için **2.000 satır** yerine ~150 satır okunuyor.

⚠ **Gövde ne zaman okunuyor:** başlık bloğu cevap vermiyorsa, ya da **değiştirilecek** dosyaysa. Tahminle kod yazılmıyor.

🚫 **Bu yöntem yorumların GÜNCEL olmasına bağlı.** Bayat bir **NEREYE** satırı ajanı yanlış dosyaya götürür — bu yüzden bir işin "*bitti*" sayılma şartına "**ilgili yorumlar güncellendi**" dahil.

Aynı mantık belgelerde de geçerli

200 sayfalık teknoloji rehberi de baştan sona okunmuyor: önce **başlık haritası** çıkarılıyor, sonra yalnızca gereken bölüm okunuyor.

Gerçek ölçüm — rehberin E.10 bölümünü okumak:

Yöntem	Okunan
Tamamını oku	6.407 satır
Önce harita, sonra bölüm	300 (başlıklar) + 204 (E.10) = 504 satır

★ **%92 tasarruf**, aynı bilgi. Bu yüzden ajana "*şu belgeyi oku*" dediğinde tamamını yutmuyor — haritaya bakıp gereken yere gidiyor.

★ SENİN KARŞILIĞIN — uzun belgede kaybolmamak

Ajan bunu terminal komutlarıyla yapıyor; **senin öğrenmene gerek yok**. VS Code'da aynı işi yapan hazır bir panel var:

İstediğin	Ajan (terminal)	★ Sen (VS Code)
Başlık haritası	<code>grep -n "^#"</code>	Outline paneli
O bölüme git	<code>sed -n '4352,4556p'</code>	Panelde başlığa tıkla

Outline paneli nasıl açılır:

Yol	Nasıl
Fareyle	Sol kenar çubuğunda Explorer (dosya ikonu) → en altta OUTLINE yazan bölümü aç
Klavyeyle	<code>Cmd+Shift+0</code> (Mac) · <code>Ctrl+Shift+0</code> (Win) → başlık listesi açılır, yazarak süz

★ 155 sayfalık rehberde "*SLA süreleri neydi*" diye ararken: `Cmd+Shift+0` → **E.4** yaz → Enter. Doğrudan oraya gider.

PDF'te okuyorsan: PDF'lerin başında **içindekiler** var, başlıklar tıklanabilir. Telefonda da çalışır.

Neden her adımda `/clear`

Konuşma uzadıkça ajanın bağlamı dolar ve ayrıntı kaybolur. Her adım kendi oturumunda yapılır; devir notu sayesinde hiçbir şey kaybolmaz.

BÖLÜM 5 — Hangi dosya ne işe yarıyor

`/yeni-proje` bittiğinde projede şunlar olur:

Kök dizin

Dosya	Ne işe yarıyor	Kim okur
<code>CLAUDE.md</code>	Ajanın uyacağı çalışma protokolü	Ajan
<code>CALISMA-KILAVUZU.md</code>	Bu dosya — projenin nasıl yürütüleceği	Sen
<code>REPO-YAPISI.md</code>	Hangi klasörde ne var, hangi iş nerede yapılıyor	İkisi
<code>README.md</code>	Projeyi kuran/çalıştıran için: kurulum, komutlar, ortam değişkenleri	Devralan geliştirici, DevOps
<code>.env.example</code>	Hangi ayarların gerektiğinin listesi — değerler boş	DevOps, devralan geliştirici
<code>.env</code>	O ayarların gerçek değerleri — 🚫 asla commit edilmez	Sadece o makine

Bu ikisi neden ayrı — somut örnek

`.env.example` git'e girer, herkes görür. Ayarların **adlarını** söyler, değerlerini değil:

```
DATABASE_URL=
JWT_SECRET=
WEB_PORT=
```

`.env` git'e **girmez**, yalnızca o makinede durur. Gerçek değerler burada:

```
DATABASE_URL=postgresql://kullanici:GercekSifre123@localhost:55432/bakim
JWT_SECRET=a8f3c91e7b2d4056
WEB_PORT=3100
```

🚫 `.env` neden commit edilmez: içinde şifre var ve **git geçmişini silinmez**. Bir kez commit edildiyse sonradan dosyayı silsen bile geçmişte durur; o depoya erişen herkes o şifreyi görebilir. Tek çözüm şifreyi değiştirmektir.

★ "Kendi değerlerini yazar" ne demek — aynı ayarın ÜÇ farklı değeri olur

Projeyi devralan biri `.env.example` 'a bakıp "demek ki şu ayarlar gerekiyormuş" der ve **kendi kurulumuna ait** değerleri yazar. Tahmin etmesi gerekmez.

Buradaki kilit nokta şu: **hiçbir ayarın "tek doğru değeri" yoktur**. Aynı değişken, çalıştığı yere göre farklı değer alır:

Değişken	Senin bilgisayarında	Deneme ortamında	Canlıda (belediyenin sunucusu)
<code>DATABASE_URL</code>	<code>postgresql://...@localhost:55432/bakim</code> — Docker'daki kap	Neon'daki <code>preview</code> dalı	Belediyenin kendi PostgreSQL sunucusu
<code>WEB_PORT</code>	<code>3100</code> (3000 doluydu)	okunmaz	<code>3000</code> — orada çakışma yok
<code>JWT_SECRET</code>	rastgele bir yerel değer	başka bir rastgele değer	tamamen başka bir rastgele değer
<code>NEXT_PUBLIC_APP_URL</code>	<code>http://localhost:3100</code>	<code>https://bakim-preview.vercel.app</code>	<code>https://bakim.izmir.bel.tr</code>

🚫 **`JWT_SECRET` satırı özellikle önemli — üç ortamda üç FARKLI değer olmak zorunda**. Aynı olursa şu olur: senin bilgisayarındaki değer bir şekilde sızarsa (ekran paylaşımı, yanlış commit, çalınan laptop) **canlı sisteme de giriş yapılabilir**, çünkü canlı da aynı sırla imzalanmış jetonları kabul eder.

⚠️ **Bu yüzden `.env.example` içine gerçek değer yazılmaz** — yazsan bile o değer kimsenin işine yaramaz, sadece riski taşımış olursun.

İstisna: Yalnızca local'de kullanılan ve **gerçekten gizli olmayan** değerler örnek dosyaya yazılabilir — Docker'daki test veritabanının `kullanici:kullanici` şifresi gibi. O şifre senin bilgisayarındaki bir kaba aittir, canlıyla hiçbir ilgisi yoktur. Yazarken yanına "yalnızca local" notu düşülür.

Bir projede hangi ayarlar çıkar — gerçek liste

⚠ Bu listenin tamamı her projede olmaz. Sol sütundaki "Ne zaman gerekir" kolonu, o satırın hangi durumda ortaya çıktığını söylüyor. "Her projede" yazanlar çekirdek; diğerleri modül eklendikçe gelir.

Ajan `.env.example` 'ı kendisi üretir ve her satırın başına yorum yazar. Bu tablo **senin ne göreceğini önceden bilmen** için.

Değer nereden gelir — dört kaynak

Aşağıdaki tablolarda geçen "Kaynak" kolonunun dört değeri var:

Kaynak	Ne demek	Nasıl elde edilir
🎲 Üretilir	Rastgele bir sır. Kimse vermiyor, sen yaratıyorsun	Terminalde <code>openssl rand -base64 32</code> yazarsın, çıkan metni yapıştırırsın
👉 Sen seçersin	Bir değer belirliyorsun; ne yazdığın senin kararın	Örn. veritabanı adını <code>bakim</code> koyarsın. Yalnızca tutarlı olması yeter
📄 Panelden kopyalanır	Bir servise üye olursun, panelinde yazar	Neon'a girersin → <i>Connection string</i> → kopyala → yapıştır
🔗 Türetilir	Başka değerlerden birleşir; elle yazılmaz	<code>DATABASE_URL</code> , kullanıcı adı + şifre + adres + veritabanı adından oluşur

★ Şunu unutma: 🎲 ve 👉 olanlar için **hiçbir yere üye olman gerekmez** — kendi bilgisayarında üretirsin. Yalnızca 📄 olanlar hesap açmayı gerektirir, ve onlar da ancak ilgili özelliği eklerken gerekir.

Çekirdek — her projede

Değişken	Ne işe yarar	Kaynak	Nasıl
<code>NODE_ENV</code>	Uygulamanın hangi modda çalıştığı	👉	<code>development</code> / <code>production</code> . Genelde araçlar kendisi ayarlar
<code>NEXT_PUBLIC_APP_URL</code>	Uygulamanın kendi adresi — e-posta linkleri, yönlendirmeler bunu kullanır	👉	Local'de <code>http://localhost:3000</code>
<code>NEXT_PUBLIC_ENV_LABEL</code>	Ekranın üstünde "DENEME ORTAMI" şeridi göstermek için	👉	<code>local</code> / <code>preview</code> / <code>production</code>

NEXT_PUBLIC_ ön eki — en kritik kural

Next.js'te bir değişkenin adı **NEXT_PUBLIC_** ile başlıyorsa, o değer **derleme sırasında tarayıcıya gönderilen JavaScript dosyasının içine gömülür**. Yani kullanıcı **F12** → *Sources* ile onu **okuyabilir**.

Ön ek	Nerede okunabilir	Ne konur
NEXT_PUBLIC_...	Tarayıcıda herkes görür	Adres, harita anahtarı (kısıtlı), ortam etiketi
(ön eksiz)	Yalnızca sunucuda	Veritabanı şifresi, JWT sırrı, API anahtarı

 **NEXT_PUBLIC_DATABASE_URL** yazmak, veritabanı şifreni siteye basmaktır. Bu, gerçekten yapılan ve gerçekten sistem düşüren bir hatadır. Kitin **env.ts** doğrulaması bu adı yakalayıp uygulamayı açılışta durdurur.

Veritabanı — veri saklayan her projede

Değişken	Ne işe yarar	Kaynak	Nasıl
POSTGRES_USER	Veritabanı kullanıcı adı		Docker'daki kap bunu okur. Local'de bakim yeter
POSTGRES_PASSWORD	Veritabanı şifresi	 local ·  canlı	Local'de basit olabilir; canlıda üretilir
POSTGRES_DB	Veritabanının adı		bakim
POSTGRES_PORT	Host makinedeki port		Çakışma varsa 55432 gibi
DATABASE_URL	Uygulamanın veritabanına bağlanma adresi	 local ·  canlı	Local'de yukarıdaki dörtten birleşir; canlıda Neon panelinden kopyalanır
DIRECT_URL	Migration için havuzsuz bağlantı		Neon kullanılıyorsa panelde ayrı adres verir. Docker'da DATABASE_URL ile aynıdır

DATABASE_URL nasıl "türetiliyor" – parçalarına ayrılmış hâli:

```
postgresql://bakim:sifre123@localhost:55432/bakim
```

```
      |      |      |      |      └─ POSTGRES_DB
      |      |      |      └─────── POSTGRES_PORT
      |      └─────────────────── sunucu adresi (local'de kendi bilgisayarın)
      └────────────────────────── POSTGRES_PASSWORD
  └──────────────────────────────── POSTGRES_USER
```

★ **Neon veya benzeri yönetilen bir veritabanı kullanıyorsan** bu satırı elle kurmazsın: panelde *Connection string* diye hazır verilir, kopyalayıp yapıştırırsın.

Portlar — aynı makinede ikinci bir proje varsa

Değişken	Ne işe yarar	Kaynak
<code>WEB_PORT</code>	Arayüzün host portu	👉 Boş bir port seçersin (<code>3100</code>)
<code>API_PORT</code>	Aynı backend varsa API'nin portu	👉 <code>4100</code>
<code>REDIS_PORT</code>	Kuyruk kullanılıyorsa	👉 <code>6479</code>

⚠ Bunlar yalnızca **senin bilgisayarındaki** kapıyı belirler. Kabin içindeki port her zaman standarttır (3000 / 4000 / 6379) ve hiç değişmez.

Kimlik doğrulama — kullanıcı girişi olan her projede

Değişken	Ne işe yarar	Kaynak	Nasıl
<code>AUTH_SECRET</code> / <code>JWT_SECRET</code>	Oturum jetonlarını imzalayan sır	🎲	<code>openssl rand -base64 32</code> · 🚫 her ortamda farklı
<code>AUTH_URL</code>	Girişten sonra dönülecek adres	👉	Uygulamanın adresi
<code>ACCESS_TOKEN_TTL</code>	Giriş jetonunun ömrü	👉	<code>15m</code> gibi
<code>REFRESH_TOKEN_TTL</code>	Yenileme jetonunun ömrü	👉	<code>7d</code> gibi
<code>GOOGLE_CLIENT_ID</code> · <code>GOOGLE_CLIENT_SECRET</code>	"Google ile giriş" varsa	📅	Google Cloud Console → <i>Credentials</i> → <i>OAuth client ID</i> oluşturursun, iki değeri kopyalarsın

Kişisel veri şifreleme — KVKK kapsamında veri tutuyorsan

Değişken	Ne işe yarar	Kaynak
<code>ENCRYPTION_KEY</code>	TC kimlik no gibi alanları veritabanında şifreli tutmak için	🎲 <code>openssl rand -base64 32</code>
<code>HASH_SALT</code>	Şifreli alanda arama yapabilmek için sabit özet tuzu	🎲

🚫 Bu iki değer kaybolursa **şifreli veriler bir daha AÇILAMAZ**. Yedeği `.env` 'den ayrı, güvenli bir yerde durmalı (parola yöneticisi).

Arka plan işleri — zamanlanmış görev veya kuyruk varsa

Değişken	Ne işe yarar	Kaynak	Nasıl
REDIS_URL	Kuyruğun (BullMQ) bağlanacağı adres	 local ·  canlı	Local'de <code>redis://localhost:6479</code> ; canlıda Upstash panelinden
CRON_SECRET	Zamanlanmış görev ucunu korur — dışarıdan tetiklenemesin		Eşleşmezse istek 401 döner

E-posta ve bildirim

Değişken	Ne işe yarar	Kaynak	Nasıl
EMAIL_API_KEY	E-posta gönderim servisinin anahtarı		Resend'e üye olursun → <i>API Keys</i> → <i>Create</i> → kopyalarsın
EMAIL_FROM	Gönderen adresi		<code>bildirim@alanadin.com</code> — ⚠ alan adının doğrulanması gerekir

Dosya yükleme — görsel/belge yüklenen her projede

Değişken	Ne işe yarar	Kaynak
BLOB_READ_WRITE_TOKEN veya S3_*	Yüklenen dosyaların saklandığı yer	 Vercel Blob / Cloudflare R2 / AWS S3 panelinden

🚫 **Yüklenen dosyalar depoya konmaz.** Git ikili dosya için tasarlanmadı; depo şişer ve geri döndürülemez.

Hata takibi — canlıya çıkan her projede

Değişken	Ne işe yarar	Kaynak
NEXT_PUBLIC_SENTRY_DSN	Hataların gönderileceği adres	 Sentry panelinden. ⚠ Gizli değil — tarayıcıya gitmek zorunda
SENTRY_AUTH_TOKEN	Derlemede kaynak haritası yüklemek için	 · 🚫 Bu gizli

Yasal — halka açık, kişisel veri işleyen projelerde

Değişken	Ne işe yarar	Kaynak
LEGAL_CONTROLLER_NAME	KVKK aydınlatma metninde görünen veri sorumlusu	 Gerçek kişi/kurum adı
LEGAL_CONTACT_EMAIL	KVKK başvurularının geleceği adres	

🚫 Depo açıksa bu ikisi **commit edilmez** — kişisel iletişim bilgisidir.

Dış servisler — projeye göre

Örnek	Ne işe yarar	Kaynak
<code>NEXT_PUBLIC_MAPBOX_TOKEN</code>	Harita gösterimi	📁 Mapbox paneli. ⚠️ Tarayıcıya iner → panelden alan adı kısıtı koy
<code>STRIPE_SECRET_KEY</code> / <code>IYZICO_*</code>	Ödeme alma	📁 Ödeme sağlayıcısının paneli
<code>SMS_API_KEY</code>	SMS gönderimi	📁 SMS sağlayıcısı

★ Bir değişken eklendiğinde ne olması gerekiyor — dört yer

Yeni bir ayar ortaya çıktığında **dört yere birden** dokunulur. Ajan bunu kendisi yapar; sen kontrol edersin:

#	Nereye	Ne yazılır
1	<code>.env.example</code>	Değişkenin adı , boş değeri ve yorum satırında ne olduğu
2	<code>.env</code>	Senin makinendeki gerçek değeri
3	<code>src/config/env.ts</code>	Zod doğrulaması — zorunlu mu, biçimi ne
4	<code>docs/project/altyapi-durumu.md</code>	Panelden alındıysa: <i>hangi hesapta, hangi ekrandan</i> alındığı

★ **3. adım neden kritik:** Doğrulama olmadan eksik bir değişken uygulamayı **çalışma anında**, kullanıcı bir işlem yaparken, anlamsız bir hatayla düşürür. Doğrulama varsa uygulama **açılıştan** durur ve net söyler: "DATABASE_URL eksik." Hatanın maliyeti saatlerden saniyelere iner.

`docs/standards/` — her projede aynı

Mühendislik kuralları: nasıl kod yazılır, hangi teknoloji kullanılır, testler nasıl olur. **Bu klasörün içeriği tüm projelerde birebir aynıdır**, çünkü kitten kopyalanıyor.

"**Elle değiştirilmez**" ne demek: Bir kuralı bu klasörde değiştirirsen yalnızca **bu projede** değişir. Bir sonraki projede eski hâliyle geri gelir — çünkü kaynak kit, bu klasör onun kopyası.

Doğru yol: Kuralı değiştirmek istiyorsan `/kit-senkron` çalıştırırsın; kural kite yazılır ve **bundan sonraki her projeye** kendiliğinden gelir.

Bir kurumun genel yönetmeliğini kendi masandaki kopyanın üstüne yazarak değiştiremezsin — merkeze bildirirsin, oradan herkese dağıtılır.

Dosya	Konu
00-stack.md	Hangi teknoloji kullanılıyor, hangisi kullanılmıyor, neden
01-architecture.md	Katmanlar, klasör yapısı, bağımlılık yönü
02-coding-standards.md	Kod ve yorum yazım kuralları
03-api-guidelines.md	API sözleşmesi, sürümleme, sayfalama
04-database.md	Veri modeli kuralları
05-auth-security.md	Kimlik doğrulama ve güvenlik
06-testing.md	Test stratejisi
08-git-workflow.md	Dal, commit, birleştirme kuralları
09-ci-cd-deploy.md	Otomatik kontroller ve yayın
11-agent-workflow.md	Ajanın nasıl çalışacağı
12-operations-and-scaling.md	Loglama, izleme, ölçekleme
13-environments.md	Local / test / canlı ayrımı
14-privacy-and-compliance.md	KVKK ve kişisel veri
15-oturum-devri.md	Oturum kapanış protokolü
(diğerleri)	Tanım listesi docs/standards/ içinde

docs/project/ — her projede var, içeriği projeye özel

⚠ "Bu projeye özel" demek "yalnızca bu projede bulunur" demek değil. Bu klasör **her projede** açılır; içindeki dosya adları da aynıdır. Değişen, **içerinin dolduruluş biçimidir**.

	docs/standards/	docs/project/
Dosya adları	Her projede aynı	Her projede aynı
İçerik	Her projede aynı	Her projede farklı
Kaynağı	Kitten kopyalanır	Görüşmeyle üretilir
Değişirse	/kit-senkron ile kite yazılır	Doğrudan düzenlenir

Yani otomatiğe bağlanan şey **yapının kendisi**: her projede aynı dokuz dosya açılır, aynı sorular sorulur, aynı sırayla doldurulur. İçlerine yazılan bilgi projeden projeye değişir.

Dosya	Hangi soruya cevap verir	Ne zaman dolar
PRD.md	Sistem ne yapacak	Kurulum Adım 3
roadmap.md	Ne yapılacak, hangi sırayla — kutucuklu	Adım 4, her adımda işaretlenir
teknoloji-ve-plan.md	Neden öyle yapıldı, teknoloji nedir	Adım 4'te açılır, her adımda büyür
decisions/	Önemli kararların gerekçesi (ADR)	Karar alındıkça
data-model.md	Kendi veri modelin	Veri modeli adımımda
integrations.md	Dış sistemlerle nasıl konuşuluyor	Entegrasyon eklendikçe
altyapi-durumu.md	Kod dışında ne yapıldı: hangi hesap açıldı, hangi panelde ne seçildi	Dış işlem yapıldıkça
sonraki-adim-prompt.md	Sırada ne var — yeni oturuma verilir	Her oturum sonunda
ogrendiklerim.md	Senin kişisel defterin: sormayı unuttuğun sorular, zor gelen konular	Ajan sorar, sen onaylarsın
CHANGELOG.md	Sürüm geçmişi	Yayın yapıldıkça

★ Dört dosya dört ayrı soruya cevap verir, karıştırılmaz:

Dosya	Cevapladığı soru
PRD.md	Sistem ne yapacak
roadmap.md	Hangi sırayla yapılacak, nerede kalındı
teknoloji-ve-plan.md	Neden öyle yapıldı, teknoloji nedir
altyapi-durumu.md	Kod dışında ne yapıldı

altyapi-durumu.md açık hâli: Bir projede kodun dışında da işler yapılır — hesap açılır, panelden ayar yapılır, alan adı satın alınır, veritabanı oluşturulur. Bunlar kodda görünmez ve **kimse hatırlamaz**.

★ Her satır üç şeyi birden yazar: **NE** yapıldı, **NEREDE** yapıldı, **NEDEN** yapıldı. "Neden" olmadan satır bir kayıt olur ama karar olmaz — altı ay sonra o ayarı değiştirmek isteyen biri neyi bozacağını bilemez.

Örnek satırlar:

2026-08-24 · Neon'da "bakim-prod" veritabanı açıldı, bölge: Frankfurt
Neden: KVKK – kişisel veri AB/Türkiye içinde kalsın diye ABD bölgesi seçilmedi. 🚫 Bölge sonradan DEĞİŞTİRİLEMEZ.

2026-08-24 · Vercel projesine DATABASE_URL değişkeni eklendi (Production)
Neden: Uygulama canlıda veritabanını bulabilsin. Değer .env'de değil panelde durur – .env dosyası sunucuya hiç gitmiyor.

2026-08-25 · Cloudflare'de bakim.izmir.bel.tr kaydı sunucu IP'sine yönlendirildi
Neden: Kullanıcı bu adresi yazınca isteğin gideceği makine belli olsun. SSL sertifikası da Cloudflare tarafından bu kayıt üzerinden veriliyor.

🚫 **Anahtar ve şifre DEĞERLERİ buraya yazılmaz** — yalnızca "şu değişken şu ortama eklendi" bilgisi yazılır. Değerler `.env` dosyasında durur.

⚠️ Bu dosya olmadan altı ay sonra "bu uyarı nereden yapmıştım" sorusunun cevabı kaybolur. Panelde yapılan bir işlemin git geçmişi yoktur.

★ Bu dosya iki proje tipinde FARKLI dolar

	Kendi projen	İşyeri projesi
Kim panel açıyor	Sen — hesabı sen açıyorsun, ödemeyi sen yapıyorsun	Kurumun DevOps ekibi
Ne yazılır	Hangi servise üye olundu, hangi bölge seçildi, hangi değişken nereye girildi	Neyin gerekli olduğu — "canlıda şu 6 değişken tanımlı olmalı"
Kim okur	Gelecekteki sen	Kuruma teslim ederken DevOps ekibi
Örnek satır	"Neon'da bakim-prod açıldı, Frankfurt"	"Uygulama şu 6 değişkeni bekliyor; DIRECT_URL migration içindir, havuzsuz olmalı"

★ **İşyeri projesinde bu dosya bir "yapıldı" defteri değil, bir "gereksinim" listesidir.** Sen paneli açmıyorsun; açacak kişiye **neyin neden gerektiğini** anlatıyorsun. Teslim paketinin en çok işe yarayan parçalarından biri budur — DevOps ekibi bu dosyayı okuyup sana soru sormadan kurulumu yapar.

BÖLÜM 5B — Canlıya çıkış: üç yol ve neyle yapıldıkları

Bu bölüm neden var: Kurulumda "*canlıya nasıl çıkacak*" sorusu geliyor. Cevap vermeden önce seçeneklerin ne olduğunu bilmen gerekiyor. Ayrıca işyeri projesinde bu iş **sana ait olmasa bile**, DevOps ekibiyle aynı dili konuşabilmek için süreci görmen gerekiyor.

Yol A — Yönetilen servisler (kendi projelerinde varsayılan)

Her katmanı ayrı bir şirket işletiyor; sen yalnızca hesap açıp bağlıyorsun. Sunucuya SSH ile girmiyorsun, işletim sistemi güncellemiyorsun.

Katman	Örnek teknoloji	Ne iş yapıyor	Neden bu katman var
Uygulama barındırma	Vercel	Next.js'i çalıştırır; her <code>git push</code> 'ta kendiliğinden yeni sürümü yayına alır	Kodun çalışacağı bir makine lazım
Veritabanı	Neon (yönetilen PostgreSQL)	Veriyi saklar, yedekler, ölçekler	Uygulama kapansa da veri kalmalı
Alan adı + DNS + SSL	Cloudflare	<code>bakim.izmir.bel.tr</code> yazınca isteğin doğru makineye gitmesi; <code>https://</code> kilidi	İnsanlar IP adresi ezberlemez
Dosya depolama	Vercel Blob / Cloudflare R2	Yüklenen görsel ve belgeler	Dosyalar veritabanında ve depoda tutulmaz
E-posta	Resend	Doğrulama ve bildirim e-postaları	Kendi sunucundan atılan mail spam'e düşer
Hata takibi	Sentry	Canlıda oluşan hataları yakalar, yığın izini gösterir	Kullanıcı hatayı bildirmez, sadece siteyi terk eder
Kuyruk (gerekliyorsa)	Upstash Redis	Arka plan işleri	Sunucusuz ortamda sürekli çalışan süreç yok

Akış:

```
git push → Vercel derler ve yayına alır → Cloudflare adresi oraya yönlendirir
      |
      ├── Neon (veritabanı)
      ├── Resend (e-posta)
      └── Sentry (hata)
```

Artısı	Eksisi
Sunucu bakımı yok — güncelleme, yama, disk hepsi onların işi	Aylık ücret her katmanda ayrı
Dakikalar içinde canlıya çıkarsın	Veri, o şirketin altyapısında durur (KVKK'da bölge seçimi kritik)
Ölçekleme kendiliğinden	Sürekli çalışan süreç barındıramazsın

Yol B — Kendi sunucun (alan adı + AWS/Hetzner + Docker Engine)

Bir bilgisayar kiralarsın, üstüne her şeyi sen kurarsın.

Katman	Teknoloji	Ne iş yapıyor
Sunucu	AWS EC2 · Hetzner · DigitalOcean	Kiralık, çıplak bir Linux makinesi
Çalıştırma	Docker Engine + Docker Compose	Kapları o makinede ayağa kaldırır
Veritabanı	Aynı sunucuda Postgres kabı veya yönetilen	Veriyi saklar
Ters vekil + SSL	Caddy veya nginx + Let's Encrypt	Gelen isteği doğru kaba yönlendirir, <code>https</code> sertifikasını alır
Alan adı	Herhangi bir kayıt firması + DNS	Adres → sunucu IP'si
Yedekleme	🚫 Senin işin — <code>pg_dump</code> + zamanlanmış görev	Yoksa disk bozulduğunda veri gider
İzleme	Uptime Kuma / Grafana	Sistem ayakta mı, ne kadar yük var

i "Docker'ı oraya mı kuruyoruz" — evet, tam olarak öyle

Kendi bilgisayarında **Docker Desktop** var: içinde Docker Engine + bir arayüz + sanal makine.

Sunucuda arayüze gerek yok, yalnızca **Docker Engine** kurulur (`apt install docker.io`).

Sonrası birebir aynı: `docker compose up -d` yazarsın, aynı `docker-compose.yml` orada da çalışır.

★ **Docker'ın asıl vaadi budur** — "benim makinemde çalışıyordu" sorununu ortadan kaldırır.

Tek fark `.env` : Portlar `3000 / 5432` olur (orada çakışma yok), şifreler gerçek olur,

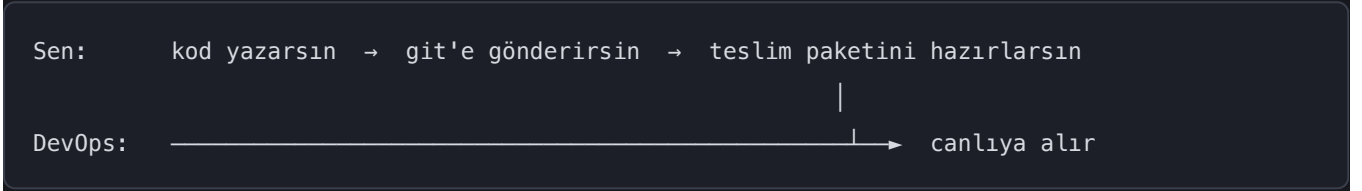
`NEXT_PUBLIC_APP_URL` alan adın olur.

Artısı	Eksisi
Veri fiziksel olarak nerede, sen biliyorsun (KVKK'da net)	❌ Güvenlik yaması, disk, yedek, izleme senin sorumluluğun
Tek fatura, yüksek yükte daha ucuz	Bir gece sunucu düşerse kaldırarak kişi sensin
Sürekli çalışan worker sorunsuz barınır	Kurulum yönetilen servislere göre kat kat uzun

⚠️ **Bu yol "daha profesyonel" değildir — daha çok sorumluluktur.** Tek kişilik bir projede yönetilen servisler genelde doğru karardır. Kurumsal ortamda ise zorunlu olabilir, çünkü verinin kurum dışına çıkması yasaktır.

Yol C — Kurumun kendi sunucusu (işyeri projelerinde en olası)

Bu yolda sen deploy yapmazsın. Görevin sınırı nettir:



Senden beklenen	Senden BEKLENMEYEN
Tek komutla ayağa kalkan <code>docker compose</code>	Alan adı satın almak
<code>.env.example</code> — hangi değişken neden gerekli	Sunucu kiralamak
<code>altyapi-durumu.md</code> — canlıda ne tanımlı olmalı	DNS ve SSL ayarı
Sağlık kontrolü ucu (<code>/health/ready</code>)	Yedekleme kurmak
Migration'ın nasıl çalıştırılacağı	Sunucuya SSH ile bağlanmak

❌ **Bu yüzden `/yeni-proje`, işyeri projesi seçildiğinde alan adı, sunucu kiralama, DNS ve SSL adımlarını hiç açmaz.** Senin ürününün son hâli "canlı bir site" değil, **kurulabilir bir pakettir.**

★ **Yine de bu bölümü okumanın sebebi:** DevOps ekibi sana "*ters vekilde websocket açık mı*", "*migration'ı biz mi koşturalım*", "*hangi portu dinliyor*" diye soracak. Süreci bilmezsen bu sorular havada kalır.

❌ **DEVOPS SINIRI — işe başlamadan sorulacak ÜÇ SORU**

İşyeri projesinde deploy senin işin değil. Ama sınırın **tam olarak nerede bittiği** çoğu kurumda yazılı değildir ve en çok karışıklık buradan çıkar.

★ **Bu üç soruyu ilk gün sor.** Cevaplar `altyapi-durumu.md` 'ye yazılır — kodda görünmezler ve altı ay sonra kimse hatırlamaz.

1. Migration'ı canlıda kim çalıştırıyor

"Veritabanı şema değişikliklerini canlıda kim çalıştırıyor — uygulama açılışta kendisi mi yapıyor, yoksa DevOps ayrı bir adımda mı?"

Bu ne demek: Veritabanına yeni bir tablo veya kolon eklendiğinde, bu değişikliği **canlı veritabanına da uygulamak** gerekir. Buna *migration* denir. İki yol var:

Yol	Nasıl	Artısı	Eksisi
A) Uygulama kendisi	Kap açılırken migration komutu çalışır	Ekstra adım yok	⚠ İki kopya aynı anda açılırsa ikisi birden çalıştırmaya kalkar
B) DevOps ayrı adımda	Yayına almadan önce elle veya hatta bir adım olarak	Kontrollü, geri alınabilir	Koordinasyon gerekir

Neyi değiştirir: `Dockerfile` 'ın başlangıç komutunu ve teslim belgesindeki yayına alma sırasını.

Cevap gelmezse: **B** varsayılır (kurumsal ortamda yaygın olan). Migration komutu README'de ayrı bir adım olarak belgelenir.

2. Gizli değerleri kim, nereye giriyor

"Veritabanı şifresi, jeton (token) anahtarı gibi gizli değerleri canlıda kim giriyor ve nereye — bir panele mi, sunucudaki dosyaya mı, yoksa kurumun bir gizli değer yönetim sistemi mi var?"

Bu ne demek: Uygulamanın çalışması için şifre ve anahtarlara ihtiyacı var. 🚫 Bunlar **koda yazılmaz** — git geçmişi silinmez, bir kez girerse orada kalır. Canlıda bir şekilde uygulamaya verilmesi gerekir.

Neyi değiştirir: `.env.example` dosyasının nasıl yazılacağını ve DevOps'a verilecek talimatı.

Cevap gelmezse: Standart yol — `.env.example` 'da **adları ve ne işe yaradıkları** belgelenir, değerleri DevOps girer. Bu her durumda çalışır.

3. Bir sürüm bozarsa geri almayı kim yapıyor

"Yayına alınan bir sürüm sorun çıkarırsa geri alma (rollback) kararını kim veriyor ve nasıl yapılıyor?"

Bu ne demek: Yeni sürüm canlıya çıktı ve bir şey bozuldu. Eski sürüme dönmek gerekiyor. ⚠ **Zor kısmı veritabanı:** kod geri alınabilir ama **migration geri alınamaz** — silinen bir kolon geri gelmez.

Neyi değiştirir: Migration'ları **geriye uyumlu** yazma zorunluluğunu. Örneğin bir kolonu silmek yerine önce kullanımdan kaldırıp bir sonraki sürümde silmek.

Cevap gelmezse: ★ **Her durumda geriye uyumlu yazılır** — bu zaten best practice. Kolon silme ve yeniden adlandırma iki aşamaya bölünür.

Hangisini ne zaman

Durum	Yol
Kendi projen, hızlı canlıya çıkmak istiyorsun	A — yönetilen
Kendi projen ama sürekli çalışan worker'ın var	A + Upstash, veya B
Veri kurum dışına çıkamaz	B veya C
İşyeri projesi, kurumun DevOps ekibi var	C — sen git'e gönderirsin
Öğrenmek istiyorsun, süreci görmek istiyorsun	B — en çok şey öğretir

★ **Karar** `docs/project/teknoloji-ve-plan.md` dosyasına gerekçesiyle yazılır, `altyapi-durumu.md` de o karara göre dolar.

BÖLÜM 6 — Hangi komut ne zaman

Komut	Ne zaman	Ne yapar
<code>/yeni-proje</code>	Projeye ilk başlarken. ★ Klasörde belge ve ayar olması sorun değil (<code>_devir/</code> , <code>.gitignore</code> , <code>.vscode/</code>); 🚫 var olan kod varsa durur ve sorar	Kurulumu baştan sona yürütür
<code>/kit-senkron</code>	Bir kuralı kalıcı hâle getirirken	Aşağıda açıldı
<code>/video-analiz <url></code>	Bir YouTube videosunda işe yarar bir şey gördüğünde	Videoyu transkriptinden inceler, kitte eksik olanı bulur ve sana onaylatır
<code>/pdf-uret <dosya.md></code>	Bir belgeyi telefonda okumak istediğinde	Karanlık temalı PDF üretir (<code>--acik</code> ile yazdırma sürümü)
<code>/clear</code>	Her roadmap adımı bitince	Sohbet geçmişini temizler, bağlam sıfırlanır
<code>claude plugin update proje-kiti@bariskose-skills</code>	Her yeni projeden önce	Kitin son sürümünü indirir
Pencere yenileme	Eklenti güncellendikten sonra	Aşağıda açıldı

★ **Dört komut da eklentiye kurar kurmaz çalışır** — `/yeni-proje` demiş olman gerekmiyor. Kurulan proje dosyaları yalnızca `/yeni-proje` ile gelir, ama komutlar en baştan hazırdır.

Pencere yenileme — nereye tıklanır

Eklenti güncellendiğinde **çalışan sürüm hâlâ eskisidir**; pencere yenilenene kadar yeni kurallar devreye girmez.

Fareyle:

1. Üst menü çubuğunda **View** (Görünüm)
2. Açılan listede **Command Palette...** (Komut Paleti)
3. Açılan kutuya yaz: **Reload Window**
4. Listede çıkan **Developer: Reload Window** satırına tıkla

Klavyeyle: **Cmd+Shift+P** (Mac) veya **Ctrl+Shift+P** (Windows) → aynı kutu açılır.

En kaba yöntem: VS Code'u tamamen kapatıp yeniden aç. Aynı işi görür.

/kit-senkron ne yapar

"Kite yaz" demenin **yapılandırılmış** hâli.

Elle söylersen ajan bir dosyaya bir şey ekler ve iş orada biter. **/kit-senkron** ise şunları sırayla yapar:

1. Projenin **docs/standards/** klasörüyle kitteki asıl kopyayı **karşılaştırır**
2. Farkları listeler: *"projede şu kural var, kitte yok"*
3. **Sana sorar:** hangileri kalıcı kural olacak, hangileri bu projeye özel
4. Seçilenleri kite yazar, sürümü yükseltir ve yayınlar
5. Kitteki iyileştirmeleri de **projeye getirir** — iki yön birden

★ Farkı şu: elle yazınca yalnızca bir dosya değişir. **/kit-senkron** ile kural **bir sonraki projede zaten orada olur**.

Kural kite yazıldı — GitHub'a kim gönderiyor

Ajan gönderiyor, sen bir şey yapmıyorsun. 4. adımdaki *"yayınlar"* kelimesi şu üç işi kapsıyor:

#	Ne oluyor	Kim yapıyor
1	Kural, kit deposundaki ilgili docs/standards/*.md dosyasına yazılır	Ajan
2	plugin.json içindeki sürüm numarası yükseltilir (1.35.1 → 1.36.0)	Ajan
3	Değişiklik commit edilip GitHub'a gönderilir	Ajan — ⚠ push öncesi sana sorar

⚠ **Push tek onay istediğin adımdır.** Kit deposu herkese açık olduğu için gönderilmeden önce ne gittiğini görmem gerekiyor. Ajan değişikliği özetler, sen *"gönder"* dersin.

Yarın başka bilgisayarda kiti nasıl güncellerim

Kit GitHub'da durduğu için **hangi makinede olduğun fark etmiyor**. Yeni makinede iki komut:

```
# 1) Kitin son sürümünü indir
claude plugin update proje-kiti@bariskose-skills

# 2) Kurulu değilse önce ekle (ilk kurulumda bir kez)
claude plugin install proje-kiti@bariskose-skills
```

Sonra **pencereyi yenile** (yukarıdaki "*Pencere yenileme*" başlığı) — yoksa çalışan sürüm hâlâ eskisidir.

★ **Kit herkese açık olduğu için giriş yapman gerekmez.** Kurumsal bir bilgisayarda kişisel GitHub hesabı bağlamak zorunda kalmazsın — bu, deponun açık tutulmasının asıl sebebi.

🚫 **Bu yüzden kite gizli hiçbir şey yazılmaz.** Ayırt edici test şu: "*Bu satırı hiç tanımadığım biri okusa, kurumum veya sistemim hakkında bir şey öğrenir mi?*" Cevap "evet" ise o bilgi kural değil **veridir**; projeye, `.env` 'e veya `altyapi-durumu.md` 'ye gider.

Üç yerdeki sürüm karışmasın

Nerede	Ne	Nasıl bakılır
GitHub	Kaynağın kendisi	Depodaki <code>plugin.json</code>
İndirilen kopya	Makinendeki önbellek	<code>claude plugin update</code> bunu tazeler
Çalışan sürüm	Açık pencerenin belleğindeki	⚠️ Reload Window yapılmadan değişmez

⚠️ En sık yaşanan kafa karışıklığı budur: güncelleme yapılır ama pencere yenilenmez, ajan eski kuralla çalışmaya devam eder.

BÖLÜM 7 — Takıldığında

Belirti	Muhtemel sebep	Ne yapılır
Ajan eski kuralla çalışıyor	Eklenti güncellendi ama pencere yenilenmedi	Reload Window
<code>/yeni-proje</code> beklenmedik davranıyor	Klasör boş değil	Fazla dosyaları taşı veya sil
Ajan cevabını bilmediğim şey soruyor	Terim açıklanmamış	" <i>Bu terimi açıkla</i> " de — kitin kuralı bunu zorunlu tutuyor
Ajan kararı bana bırakıyor	Mühendislik seçimini devretmiş	" <i>Sen karar ver, gerekçesini söyle</i> " de
Nerede kaldığımı hatırlamıyorum	Kutucuk işaretlenmemiş	<code>roadmap.md</code> ve <code>sonraki-adım-prompt.md</code> 'ye bak
Cevaplar yüzeyselleşti	Bağlam dolmuş	<code>/clear</code> yap, devir notuyla devam et

BÖLÜM 8 — Öğrendiklerim defteri

`docs/project/ogrendiklerim.md` senin kişisel defterin. Kite yazılacak kadar genel olmayan ama unutulursa bedeli tekrar ödenecek notlar buraya girer.

Kim yazar

Ajan sorar, sen onaylarsın. Oturum kapanırken şuna benzer bir soru gelir:

"*Bu oturumda bağımlılığın tersine çevrilmesi konusunu üçüncü örnekte oturttuğunu fark ettim. Deftere yazayım mı?*"

🚫 Senin "*bunu deftere yaz*" demen beklenmiyor. O sırada zaten öğrenmekle meşgulsün; not almayı hatırlaman beklenemez. **Fark eden taraf teklif eder.**

Ne nereye gider

Gözlem	Nereye
"Bu terimi ilk kez anladım"	Defter → zor gelen kararlar
"Şunu sormayı unutmuşuz"	Defter → sormayı unuttuğum sorular
Aynı hata ikinci kez	Defter → tekrar eden hatalar
Aynı hata üçüncü kez	Artık kişisel değil → kite taşınır
"Her projede böyle yapılmalı"	Doğrudan kite

★ **Ayrım basit:** Kite **kural** gider — "şu durumda şu yapılır." Deftere **deneyim** girer — "ben şunu atlamıştım." Bir deneyim üç kez tekrarlanırsa kurala dönüşür ve kite taşınır.

Ne işe yarıyor

İki şey:

1. **Bir sonraki projenin kontrol listesi.** "Geçen sefer eş zamanlı kullanıcı sayısını sormayı unutmuşum" notu, bu sefer sorulmasını sağlar
2. **İlerlemenin ölçüsü.** "Zor gelen kararlar" listesi zamanla kısalır. Kısalması öğrendiğinin kanıtıdır

★ Ajan seviyeni takip eder — dört seviyeli defter

`docs/project/ogrendiklerim.md` içinde bir **seviye defteri** var. On konu alanı (mimari · backend · frontend · veritabanı · test · DevOps · güvenlik · mobil · AI ile çalışma), her biri 0–3 arası ölçülüyor.

Sv	Ne demek	Ajan ne yapıyor
0	İlk kez geçiyor	Üç adımda tam açar
1	Gördün, soru sordun	Kısa hatırlatma
2	Soru sormadan izledin	Terimi kullanır, tek cümle hatırlatır
3	★ Sahipleniyorsun	Doğrudan kullanır, açıklamaz

🚫 "Tamam" demek seviye yükseltmez

En önemli kural bu. Seviye yalnızca **senin ürettiğin** bir şeyle yükselir:

Sen ne yaptın	Sayılır mı
★ Ajanı o konuda düzelttin	En güçlü kanıt — anlamadan düzeltemezsin
★ Terimi kendi cümlede kullandın	Güçlü
Sonucunu sordun ("o zaman şu olmaz mı?")	Güçlü
Okudun, soru sormadın	⚠ Zayıf — tek başına yetmez
"Tamam" dedin	🚫 Sayılmaz

🚫 **Tek gözlemle de yükselmez** — en az **iki ayrı oturumda** kanıt gerekir. Sonra ajan sana **sorar**, kendiliğinden yazmaz.

⚠ **Seviye düşer de — bu normal**

Durum	Etki
Konu 8 haftadır geçmedi	Bir seviye düşer
"Bunu tekrar açıkla" dersin	★ Doğrudan 0
O konuda yanlış bir şey söylersen	Bir seviye düşer

🚫 Düşme başarısızlık değil. Full-stack + mobil + DevOps + test + veritabanı hepsinin oturması **3–6 ay** sürer; bu sürede unutmak kaçınılmazdır.

★ **Kavram ile kelime ayrı ölçülür**

Bir kavramı anlamış olabilirsin ama anlatımda geçen bir kelimeyi bilmiyorsundur. Bu **ayrı bir eksik** — kavramın seviyesini düşürmez. Kelimeyi sorduğunda ajan onu **Kelime defteri**'ne yazar, seviyene dokunmaz.

Bir yanlışın olursa nasıl düzeltilir

Ajan **kişiselleştirmez** ama ★ **hatanın yerini mutlaka gösterir** — yoksa aynı yanlışını tekrar yaparsın:

#	Ne söyler
1	Ne söyledin — alıntılar
2	Doğrusu ne
3	★ Ayırım tam olarak nerede
4	Varsa neden karıştırıldığı

★ Üçüncüsü en önemlisi: tekrarı önleyen şey doğru cevap değil, **ayırım noktası**.

Öğretmeyi ne zaman bırakıyor

Durum	Ajan
Konu seviye 3, 8 haftadır düşmedi	O konuda anlatım durur
Bir alanın tamamı seviye 3	Yalnızca yeni şeyler anlatılır
Tüm alanlar seviye 3	★ Yalnızca karar ve gerekçe sunulur

★ **Neden gerekli:** Bu kılavuzun ve rehberlerin her terimi açması, başlarken doğru — ama altıncı projede her **transaction** kelimesinde kurabiye kalıbı benzetmesi okumak zaman kaybı olur. Liste, anlatımın **seninle birlikte büyümesini** sağlıyor.

⚠ **Bu liste kite gitmez, projede kalır** — ama **/yeni-proje** yeni projeye başlarken bir öncekinden **kopyalar**. Böylece seviyen projeler arasında taşınır, her seferinde sıfırdan başlamaz.

BÖLÜM 9 — Bu kılavuz nasıl kısaltılır

Amaç bu dosyaya bağımlı kalmak değil.

- **Terimler oturduğunda** → Bölüm 1'i sil
- **Akış ezberlendiğinde** → Bölüm 3'ü kısalt, yalnızca komut kalsın
- **Dosya haritası aklında kaldığında** → Bölüm 5'i sil

Kalması gereken tek bölüm: **Bölüm 4 — oturum ritmi**. O, ezberlenmesi değil her seferinde uygulanması gereken bir kontrol listesidir.